

# FastAPI for AI Engineering — Intro Video Narration Script

Target runtime: ~5 minutes Word count: ~660 words

---

## Slide 1 — This Series, In One Glance

~25 sec

Turn Python into a real API for an AI application — one lab at a time. That's this series, in one line. Across thirteen labs plus a capstone, you'll build what a real AI backend needs — validated endpoints, file uploads, streaming, real-time chat, security, background tasks — all on one FastAPI foundation. No prior experience required. We start from zero.

---

## Slide 2 — What is FastAPI?

~85 sec

First, understand what an API is. An API is an interface on your server. Your app exposes endpoints — specific URLs — and other programs call them over HTTP: they send a request, you send back a response. That's the whole idea.

You could write that request-and-response handling by hand. FastAPI does it for you — it receives the request, routes it to the right Python function, runs it, and returns the response. And it gives you interactive docs for free, at a URL called /docs, where you can test your API straight from the browser.

Why FastAPI specifically? Because it's built for exactly what this series builds — chat, uploads, streaming, real-time features, auth — which is why it's one of the most popular choices for AI backends.

So hold this picture in mind: request comes in, FastAPI routes it, your function runs, a response goes back. Every lab from here is just a new way of doing that one loop.

---

## Slide 3 — The Roadmap

~65 sec

Here's the roadmap, and it's worth understanding the order.

We start with Foundations — validation, error handling, dependency injection — because before your API does anything real, it has to trust its inputs.

Once inputs are safe, we move to Async and Real-time I/O — file uploads, streaming answers, WebSockets, background tasks — this is where AI apps start to feel alive.

Then Production-grade APIs — security, rate limiting, observability, testing — because a working demo isn't a real product yet.

And finally, Synthesis: everything together, first in a guided multi-tenant platform, then in your own capstone, unguided.

Same foundation the whole way — each stage just asks it to do one more real-world thing.

---

## **Slide 4 — Anatomy of One Lab**

~35 sec

Every lab follows the same shape, so once you've done one, you know how to do them all.

Each one gives you an article explaining the concept, a notebook you run cell by cell — the first cell installs everything — and that same code as a plain .py script, for when you want a real server instead.

Then an assignment: a simple, step-by-step task you build yourself. Read, run, build — then the next lab.

---

## **Slide 5 — How to Run the Labs**

~45 sec

Quick but important — before you touch Lab 1, open the reference file.

It covers how to actually run these labs: in Colab, Jupyter, VS Code, and the other path — taking that .py script and running it as a real server with Uvicorn, testing it through the docs page instead of a notebook.

You don't need the server route on day one — the notebook is the default, zero setup. But this is the one document you'll keep coming back to all series. Don't skip it — open it now, skim it, and keep it bookmarked.

---

## **Slide 6 — Getting Started**

~35 sec

Last thing — these labs aren't a rigid ladder. Each one states what it assumes, so several can be done out of order.

If you're not sure where to start: skim the reference file, do Lab 1, then the lighter early labs — exception handling, dependency injection — before the heavier async and file-upload lab after them. Finish with the multi-tenant platform, then build your own capstone.

Alright — let's get started.

